

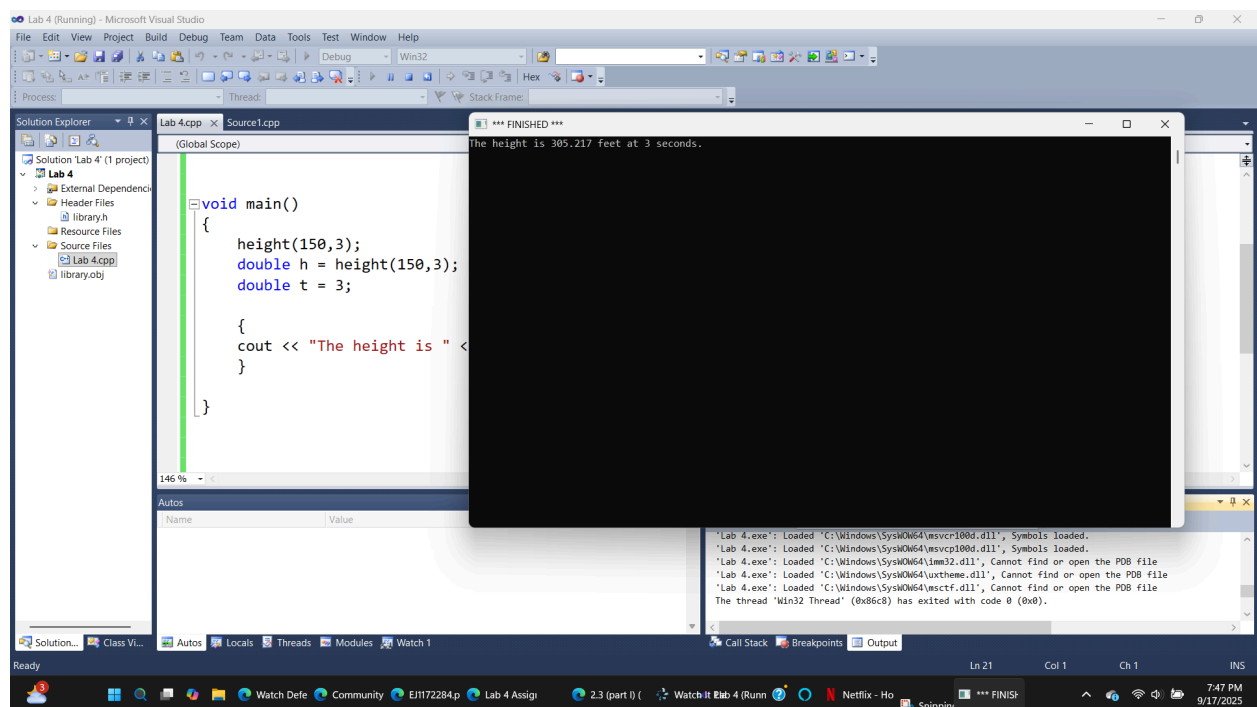
1. The Calculation

```
#include "library.h"
```

```
double height(double v, double t)
{
    double G = 32.174;
    double h = v*t-0.5*G*t*t;
    return h;
}
```

```
void main()
{
    height(150,3);
    double h = height(150,3);
    double t = 3;

    {
        cout << "The height is " << h << " feet at " << t << " seconds." << endl;
    }
}
```



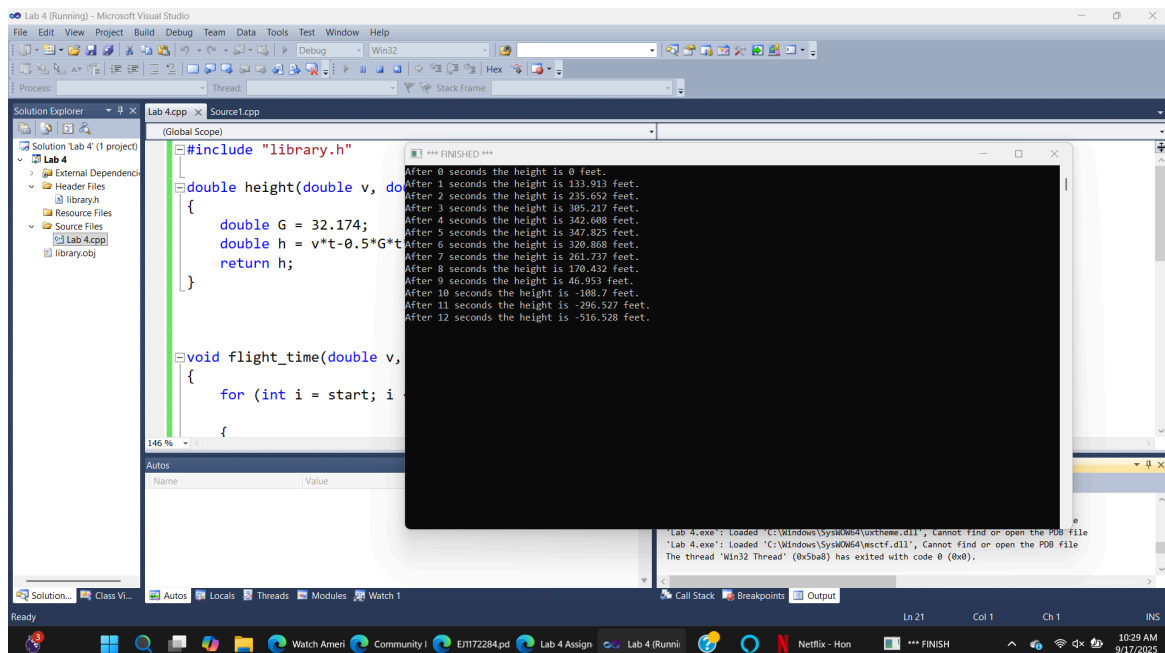
2. Tabulation

```
#include "library.h"
```

```
double height(double v, double t)
{
    double G = 32.174;
    double h = v*t-0.5*G*t*t;
    return h;
}
```

```
void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(150,i);
        cout << "After " << i << " seconds the height is " << h << " feet." << endl;
    }
    cout << endl;
}
```

```
void main()
{
    flight_time(150,0,12);
}
```



3. Interaction

```
#include "library.h"

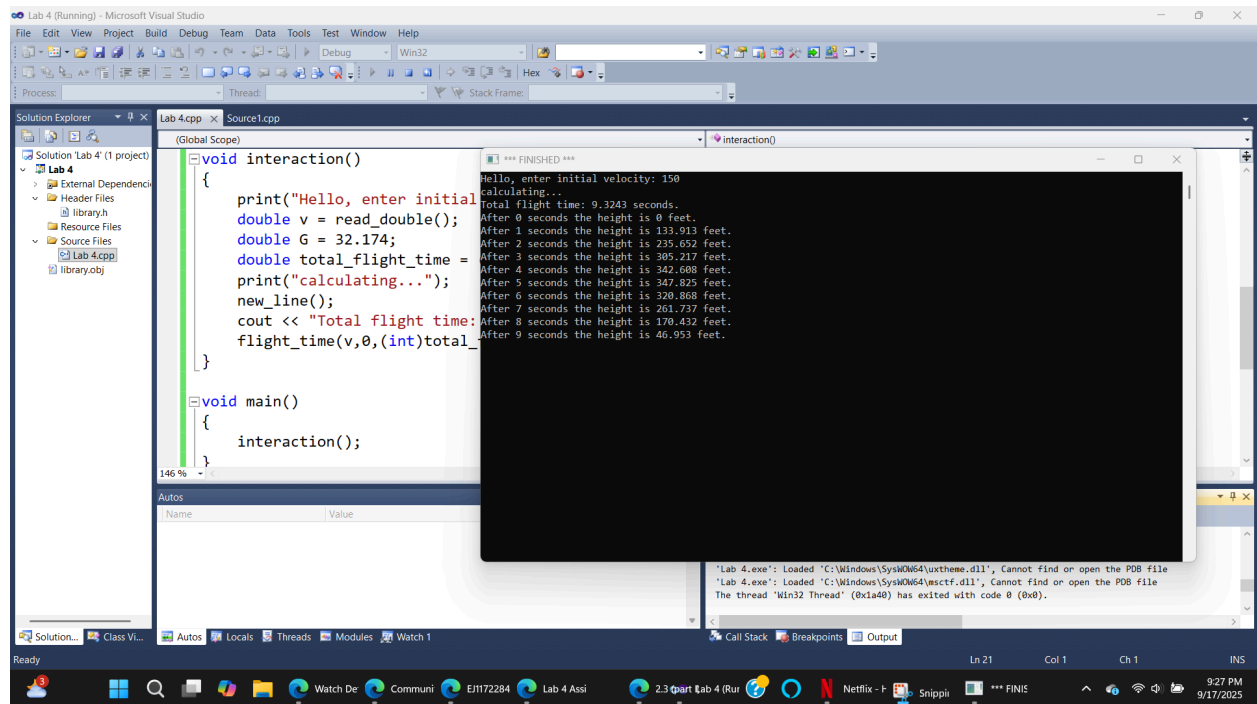
double G = 32.174;

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
        cout << "After " << i << " seconds the height is " << h << " feet." << endl;
    }
    cout << endl;
}

void interaction()
{
    print("Hello, enter initial velocity: ");
    double v = read_double();
    double G = 32.174;
    double total_flight_time = (2 * v) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    flight_time(v,0,(int)total_flight_time);
}

void main()
{
    interaction();
}
```



4. Visual Representation

```
#include "library.h"

double G = 32.174;

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
        cout << "After " << i << " seconds the height is " << h << " feet." << endl;
    }
    cout << endl;
}

void interaction()
{
    print("Hello, enter initial velocity: ");
    double v = read_double();
    double G = 32.174;
    double total_flight_time = (2 * v) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    flight_time(v,0,(int)total_flight_time);
}

void visual(double v)
{
    double G = 32.174;
    double total_flight_time = (2 * v) / G;
    int t = 0;
    while (t <= (int)total_flight_time)
    {
        double h = height(v,t);
        double ratio = (double)t / total_flight_time;
```

[illegible]

5. First Flight

```
#include "library.h"

double G = 32.174;

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
        cout << "After " << i << " seconds the height is " << h << " feet." << endl;
    }
    cout << endl;
}

void interaction()
{
    print("Hello, enter initial velocity: ");
    double v = read_double();
    double G = 32.174;
    double total_flight_time = (2 * v) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    flight_time(v,0,(int)total_flight_time);
}

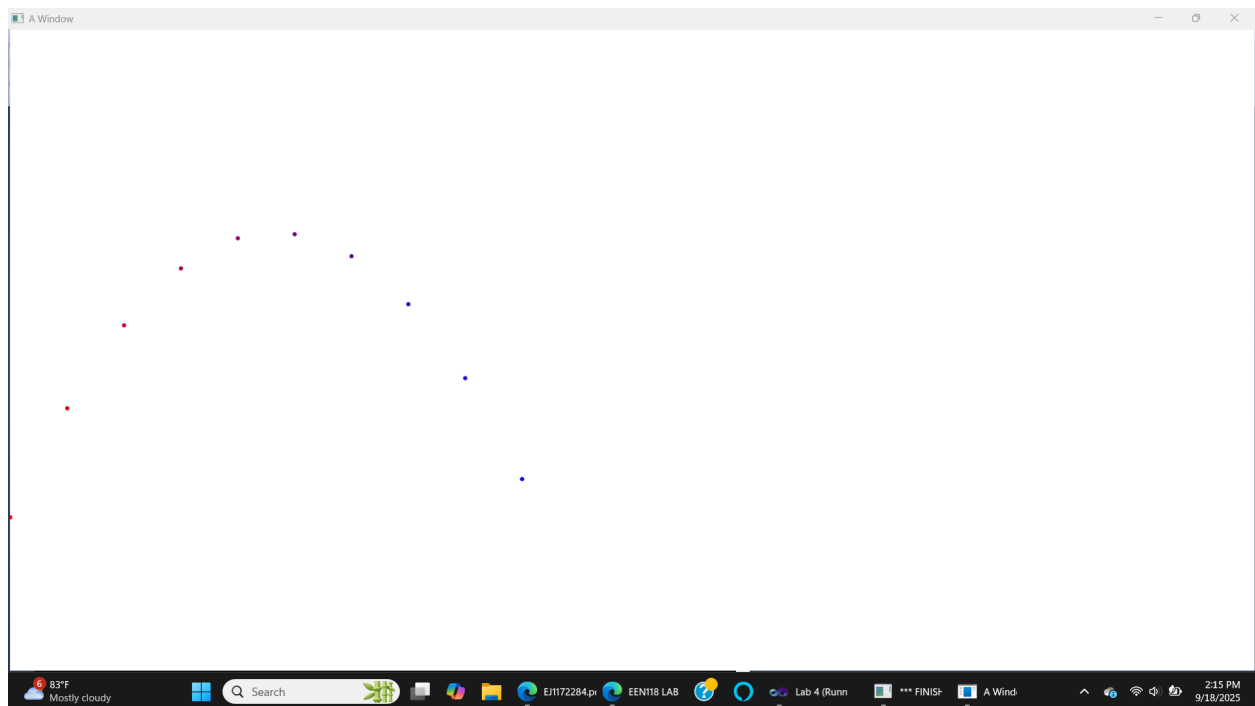
void visual(double v)
{
    double G = 32.174;
    double total_flight_time = (2 * v) / G;
    int t = 0;
    while (t <= (int)total_flight_time)
    {
        double h = height(v,t);
        double ratio = (double)t / total_flight_time;
```

```

        double R = 1.0 - ratio;
double G = 0.0;
double B = ratio;
        double x = t * 70;
double y = 600 - h;
        set_pen_width(5);
        set_pen_color(R, G, B);
draw_point((int)x, (int)y);
        t = t + 1;
    }
}

void main()
{
    print("Hello, enter initial velocity: ");
double v = read_double();
cout << "Drawing visual representation..." << endl;
    make_window(900,400);
    visual(v);
}

```



6. A Nice Arc

```
#include "library.h"

double G = 32.174;

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
        cout << "After " << i << " seconds the height is " << h << " feet." << endl;
    }
    cout << endl;
}

void interaction()
{
    print("Hello, enter initial velocity: ");
    double v = read_double();
    double G = 32.174;
    double total_flight_time = (2 * v) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    flight_time(v,0,(int)total_flight_time);
}

void visual(double v)
{
    double total_flight_time = (2 * v) / G;
    double t = 0;
    double step = 0.01;

    while (t <= (int)total_flight_time)
    {
```

The screenshot displays the Microsoft Visual Studio IDE with a C++ project named 'Lab 4'. The main window shows the source code for 'Source1.cpp', which includes a function to calculate the flight time of an object launched at an initial velocity. The code is as follows:

```

#include <iostream>
using namespace std;

double flight_time(v,0,(int)total_flight_time);

void vis
{

```

A window titled 'A Window' is open, showing a graph of the projectile's trajectory. The trajectory is a parabolic curve starting at the origin (0,0) and ending at a point on the x-axis. The curve is colored red and blue. The graph is titled 'A Window' and has a standard Windows window frame.

The console window at the bottom shows the program's output:

```

*** FINISHED ***
Hello, enter initial velocity: 150
Drawing visual representation...

```

The Solution Explorer on the left shows the project structure for 'Lab 4', including 'External Dependencies', 'Header Files', 'library', 'Resource Files', and 'Source Files'. The 'Source Files' folder contains 'Lab 4.cpp'.

The bottom status bar shows the current line (Ln 21), column (Col 1), and character (Ch 1). The system tray at the bottom right shows the date and time: 2:46 PM, 9/18/2023.

7. **Take the Battle to the Enemy**

```
#include "library.h"

double G = 32.174;

void draw_circle(double radius, const int steps)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    for (int i = 0; i < steps; i++)
    {
        draw_distance(step_length);
        turn_right_by_degrees(turn_angle);
    }
}

void draw_cannon(double xg, double yg, double a)
{
    double r = 30;
    double L1 = 40;
    double L2 = 80;
    double w1 = 40;
    double w2 = 20;

    double xc = xg;
    double yc = yg - r;

    move_to(xc, yc);
    draw_circle(r, 60);

    double b = asin((w1 - w2)/2/(L1 + L2));

    double xp = xc - L1 * sin(a-b);
    double yp = yc + L1 * cos(a-b);

    double len = (L1 + L2) * cos(b);

    double d = sqrt(len*len + (w1*w1)/4);
```

```

double g = asin((w1/2)/d);
double xe = xp + d * sin(a-g);
double ye = yp - d * cos(a-g);

double phi = a - g;

double px = cos(phi);
double py = sin(phi);

double back_top_x = xp + (w1/2) * px;
double back_top_y = yp + (w1/2) * py;
double back_bot_x = xp - (w1/2) * px;
double back_bot_y = yp - (w1/2) * py;

double front_top_x = xe + (w2/2) * px;
double front_top_y = ye + (w2/2) * py;
double front_bot_x = xe - (w2/2) * px;
double front_bot_y = ye - (w2/2) * py;

move_to(back_bot_x, back_bot_y);
draw_to(front_bot_x, front_bot_y);
draw_to(front_top_x, front_top_y);
draw_to(back_top_x, back_top_y);
draw_to(back_bot_x, back_bot_y);
}

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
    }

    cout << endl;
}

void interaction()

```

```

{
    print("Hello, enter initial velocity: ");
    double v = read_double();
    double total_flight_time = (2 * v) / G;
    double t = 1.1;
    print("Enter cannon angle (degrees): ");
    double a = read_double();
    double d = v * t * sin(a);
    double d_final = (2 * v * v * cos(a) * sin(a)) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    cout << "Final horizontal position: " << d_final << endl;
    flight_time(v,0,(int)total_flight_time);
}

```

```

void visual(double v, double a)

```

```

{
    cout << "Drawing visual representation..." << endl;
    double total_flight_time = (2 * v) / G;
    double pi = 3.141592653589793;
    double t = 1.1;
    double step = 0.01;

    double a_radians = a * pi / 180;

    while (t <= (int)total_flight_time)
    {
        double h = v * t * sin(a_radians) - 0.5 * G * t * t;
        double ratio = (double)t / total_flight_time;
        double R = 1.0 - ratio;

        double G = 0.0;
        double B = ratio;
        double x = t * 90;
        double y = 400 - h;

        double d = v * t * cos(a_radians);

        set_pen_width(5);
        set_pen_color(R, G, B);
        draw_point((int)x, (int)y);
        t = t + step;
    }
}

```

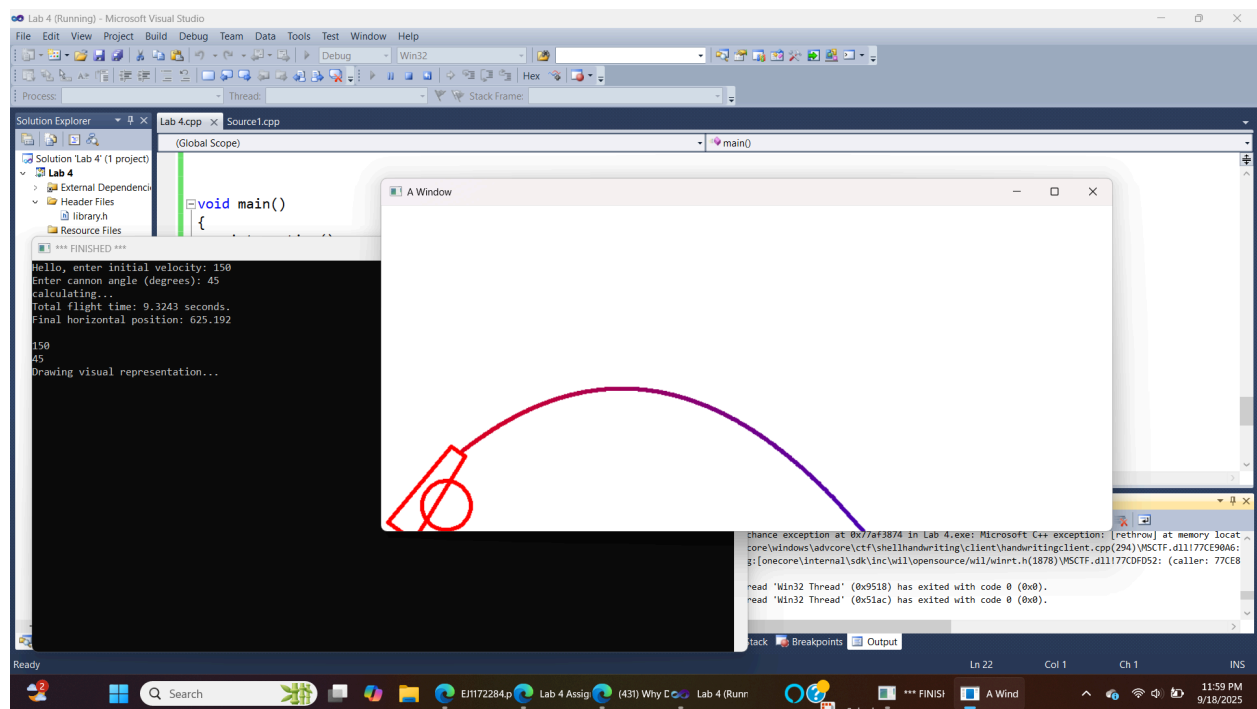
```
}
```

```
void main()
```

```
{
```

```
    interaction();  
    double v = read_double();  
    double a = read_double();  
    double pi = 3.141592653589793;  
    double a_radians = a * pi / 180;  
    make_window(900,400);  
    visual(v,a);  
    set_pen_color(color::red);  
    draw_cannon(50,400,a_radians);
```

```
}
```



8. The Game

```
#include "library.h"
```

```
double G = 32.174;
```

```
void draw_circle(double radius, const int steps)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    for (int i = 0; i < steps; i++)
    {
        draw_distance(step_length);
        turn_right_by_degrees(turn_angle);
    }
}
```

```
void draw_cannon(double xg, double yg, double a)
{
    double r = 30;
    double L1 = 40;
    double L2 = 80;
    double w1 = 40;
    double w2 = 20;

    double xc = xg;
    double yc = yg - r;

    move_to(xc, yc);
    draw_circle(r, 60);

    double b = asin((w1 - w2)/2/(L1 + L2));

    double xp = xc - L1 * sin(a-b);
    double yp = yc + L1 * cos(a-b);

    double len = (L1 + L2) * cos(b);

    double d = sqrt(len*len + (w1*w1)/4);
    double g = asin((w1/2)/d);
```

```

double xe = xp + d * sin(a-g);
double ye = yp - d * cos(a-g);

double phi = a - g;

double px = cos(phi);
double py = sin(phi);

double back_top_x = xp + (w1/2) * px;
double back_top_y = yp + (w1/2) * py;
double back_bot_x = xp - (w1/2) * px;
double back_bot_y = yp - (w1/2) * py;

double front_top_x = xe + (w2/2) * px;
double front_top_y = ye + (w2/2) * py;
double front_bot_x = xe - (w2/2) * px;
double front_bot_y = ye - (w2/2) * py;

move_to(back_bot_x, back_bot_y);
draw_to(front_bot_x, front_bot_y);
draw_to(front_top_x, front_top_y);
draw_to(back_top_x, back_top_y);
draw_to(back_bot_x, back_bot_y);
}

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
    }

    cout << endl;
}

void interaction()
{

```

```

    print("Hello, enter initial velocity: ");
    double v = read_double();
    double total_flight_time = (2 * v) / G;
    double t = 0;
    print("Enter cannon angle (degrees): ");
    double a = read_double();
    double d = v * t * sin(a);
    double d_final = (2 * v * v * cos(a) * sin(a)) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    cout << "Final horizontal position: " << d_final << endl;
    flight_time(v,0,(int)total_flight_time);
}

void visual(double v, double a, double t)
{
    cout << "Drawing visual representation..." << endl;
    double total_flight_time = (2 * v) / G;
    double pi = 3.141592653589793;
    double step = 0.01;

    double a_radians = a * pi / 180;

    while (t <= (int)total_flight_time)
    {
        double h = v * t * sin(a_radians) - 0.5 * G * t * t;
        double ratio = (double)t / total_flight_time;
        double R = 1.0 - ratio;

        double G = 0.0;
        double B = ratio;

        double d = v * t * cos(a_radians);

        double x = 50 + d;
        double y = 400 - h;

        set_pen_width(5);
        set_pen_color(R, G, B);
        draw_point((int)x, (int)y);
        t = t + step;
    }
}

```

```

void enemy_HQ(const int x, const int y)
{
    move_to(x,y);
    set_pen_width(1);
    draw_distance(80);
    turn_right_by_degrees(90);
    draw_distance(80);
    turn_right_by_degrees(90);
    draw_distance(80);
    turn_right_by_degrees(90);
    draw_distance(80);
    turn_right_by_degrees(90);
    move_to(x + 10,y - 50);
    draw_distance(10);
    turn_right_by_degrees(90);
    draw_distance(20);
    turn_right_by_degrees(90);
    draw_distance(10);
    turn_right_by_degrees(90);
    draw_distance(20);
    turn_right_by_degrees(90);
    move_to(x + 50,y - 50);
    draw_distance(10);
    turn_right_by_degrees(90);
    draw_distance(20);
    turn_right_by_degrees(90);
    draw_distance(10);
    turn_right_by_degrees(90);
    draw_distance(20);
    turn_right_by_degrees(90);
}

```

```

void main()
{
    interaction();
    double v = read_double();
    double a = read_double();
    double pi = 3.141592653589793;
    double a_radians = a * pi / 180;
    make_window(1000,400);

    visual(v+20, a+20, 0.66);
}

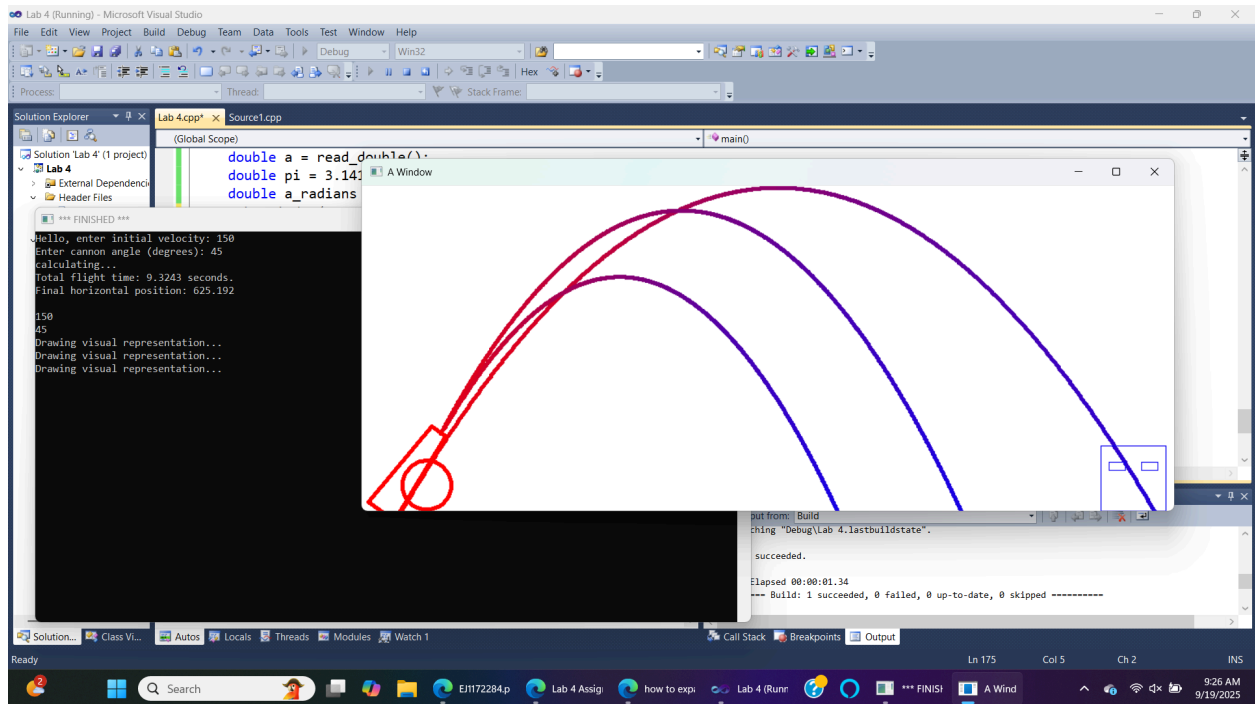
```

```

visual(v, a+20, 0.82);
visual(v+35, a+14.75, 0); // hit shot

enemy_HQ(910,400);
set_pen_width(5);
set_pen_color(color::red);
draw_cannon(50,400,a_radians);
}

```



9. Enemy Defences

```
#include "library.h"
```

```
double G = 32.174;
```

```
void draw_circle(double radius, const int steps)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    for (int i = 0; i < steps; i++)
    {
        draw_distance(step_length);
        turn_right_by_degrees(turn_angle);
    }
}
```

```
void draw_cannon(double xg, double yg, double a)
{
    double r = 30;
    double L1 = 40;
    double L2 = 80;
    double w1 = 40;
    double w2 = 20;

    double xc = xg;
    double yc = yg - r;

    move_to(xc, yc);
    draw_circle(r, 60);

    double b = asin((w1 - w2)/2/(L1 + L2));

    double xp = xc - L1 * sin(a-b);
    double yp = yc + L1 * cos(a-b);

    double len = (L1 + L2) * cos(b);

    double d = sqrt(len*len + (w1*w1)/4);
    double g = asin((w1/2)/d);
```

```

double xe = xp + d * sin(a-g);
double ye = yp - d * cos(a-g);

double phi = a - g;

double px = cos(phi);
double py = sin(phi);

double back_top_x = xp + (w1/2) * px;
double back_top_y = yp + (w1/2) * py;
double back_bot_x = xp - (w1/2) * px;
double back_bot_y = yp - (w1/2) * py;

double front_top_x = xe + (w2/2) * px;
double front_top_y = ye + (w2/2) * py;
double front_bot_x = xe - (w2/2) * px;
double front_bot_y = ye - (w2/2) * py;

move_to(back_bot_x, back_bot_y);
draw_to(front_bot_x, front_bot_y);
draw_to(front_top_x, front_top_y);
draw_to(back_top_x, back_top_y);
draw_to(back_bot_x, back_bot_y);
}

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)
    {
        double h = height(v,i);
    }

    cout << endl;
}

void interaction()
{

```

```

    print("Hello, enter initial velocity: ");
    double v = read_double();
    double total_flight_time = (2 * v) / G;
    double t = 0;
    print("Enter cannon angle (degrees): ");
    double a = read_double();
    double d = v * t * sin(a);
    double d_final = (2 * v * v * cos(a) * sin(a)) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    cout << "Final horizontal position: " << d_final << endl;
    flight_time(v,0,(int)total_flight_time);
}

void visual(double v, double a, double t_start, double t_end)
{
    cout << "Drawing visual representation..." << endl;
    double total_flight_time = (2 * v) / G;
    double pi = 3.141592653589793;
    double step = 0.01;
    double t = t_start;

    double a_radians = a * pi / 180;

    while (t <= t_end && t <= (int)total_flight_time)
    {
        double h = v * t * sin(a_radians) - 0.5 * G * t * t;
        double ratio = (double)t / total_flight_time;
        double R = 1.0 - ratio;

        double G = 0.0;
        double B = ratio;

        double d = v * t * cos(a_radians);

        double x = 50 + d;
        double y = 400 - h;

        set_pen_width(5);
        set_pen_color(R, G, B);
        draw_point((int)x, (int)y);
        t = t + step;
    }
}

```

```
void enemy_HQ(const int x, const int y)
```

```
{  
    move_to(x,y);  
    set_pen_width(1);  
    draw_distance(80);  
    turn_right_by_degrees(90);  
    draw_distance(80);  
    turn_right_by_degrees(90);  
    draw_distance(80);  
    turn_right_by_degrees(90);  
    draw_distance(80);  
    turn_right_by_degrees(90);  
    move_to(x + 10,y - 50);  
    draw_distance(10);  
    turn_right_by_degrees(90);  
    draw_distance(20);  
    turn_right_by_degrees(90);  
    draw_distance(10);  
    turn_right_by_degrees(90);  
    draw_distance(20);  
    turn_right_by_degrees(90);  
    move_to(x + 50,y - 50);  
    draw_distance(10);  
    turn_right_by_degrees(90);  
    draw_distance(20);  
    turn_right_by_degrees(90);  
    draw_distance(10);  
    turn_right_by_degrees(90);  
    draw_distance(20);  
    turn_right_by_degrees(90);  
}  

```

```
void enemy_defences(const int x, const int y)
```

```
{  
    set_pen_width(5);  
    move_to(x,y);  
    draw_distance(160);  
}  

```

```
void main()
```

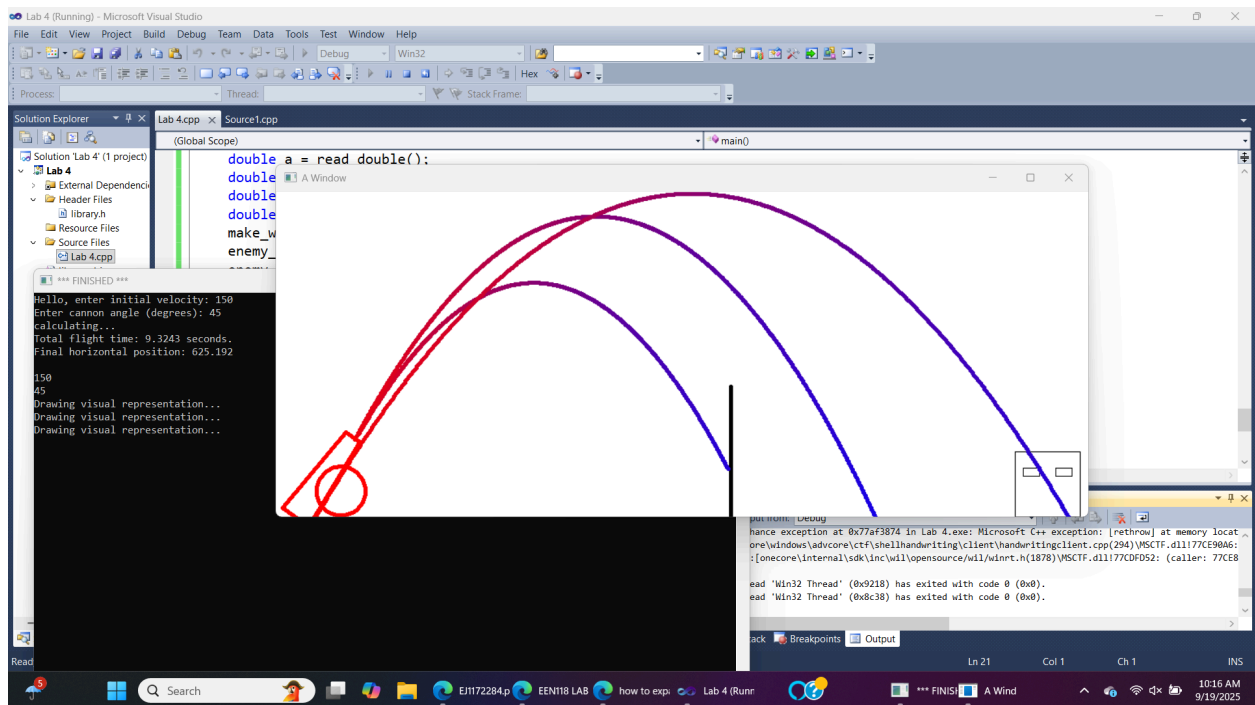
```

{
    interaction();
    double v = read_double();
    double a = read_double();
    double pi = 3.141592653589793;
    double a_radians = a * pi / 180;
    double total_flight_time = (2 * v) / G;
    make_window(1000,400);
    enemy_HQ(910,400);
    enemy_defences(560,400);

    visual(v, a+20, 0.82, 8);
    visual(v+20, a+20, 0.66, 15);
    visual(v+35, a+14.75, 0, 15); // hit shot

    set_pen_width(5);
    set_pen_color(color::red);
    draw_cannon(50,400,a_radians);
}

```



10. Make it Interesting

```
#include "library.h"

double G = 32.174;

void draw_circle(double radius, const int steps)
{
    double pi = acos(-1.0);
    double circumference = 2 * pi * radius;
    double step_length = circumference/steps;
    double turn_angle = 360/steps;

    for (int i = 0; i < steps; i++)
    {
        draw_distance(step_length);
        turn_right_by_degrees(turn_angle);
    }
}

void draw_cannon(double xg, double yg, double a)
{
    double r = 30;
    double L1 = 40;
    double L2 = 80;
    double w1 = 40;
    double w2 = 20;

    double xc = xg;
    double yc = yg - r;

    move_to(xc, yc);
    draw_circle(r, 60);

    double b = asin((w1 - w2)/2/(L1 + L2));

    double xp = xc - L1 * sin(a-b);
    double yp = yc + L1 * cos(a-b);

    double len = (L1 + L2) * cos(b);

    double d = sqrt(len*len + (w1*w1)/4);
    double g = asin((w1/2)/d);
```

```

double xe = xp + d * sin(a-g);
double ye = yp - d * cos(a-g);

double phi = a - g;

double px = cos(phi);
double py = sin(phi);

double back_top_x = xp + (w1/2) * px;
double back_top_y = yp + (w1/2) * py;
double back_bot_x = xp - (w1/2) * px;
double back_bot_y = yp - (w1/2) * py;

double front_top_x = xe + (w2/2) * px;
double front_top_y = ye + (w2/2) * py;
double front_bot_x = xe - (w2/2) * px;
double front_bot_y = ye - (w2/2) * py;

move_to(back_bot_x, back_bot_y);
draw_to(front_bot_x, front_bot_y);
draw_to(front_top_x, front_top_y);
draw_to(back_top_x, back_top_y);
draw_to(back_bot_x, back_bot_y);
}

double height(double v, double t)
{
    double h = v*t-0.5*G*t*t;
    return h;
}

void flight_time(double v, int start, int end)
{
    for (int i = start; i <= end; i++)

        {
            double h = height(v,i);
        }

    cout << endl;
}

void interaction()
{

```

```

    print("Hello, enter initial velocity: ");
    double v = read_double();
    double total_flight_time = (2 * v) / G;
    double t = 0;
    print("Enter cannon angle (degrees): ");
    double a = read_double();
    double d = v * t * sin(a);
    double d_final = (2 * v * v * cos(a) * sin(a)) / G;
    print("calculating...");
    new_line();
    cout << "Total flight time: " << total_flight_time << " seconds." << endl;
    cout << "Final horizontal position: " << d_final << endl;
    flight_time(v,0,(int)total_flight_time);
}

void game(double v, double a, double t_start, double t_end)
{
    cout << "Drawing visual representation..." << endl;
    double total_flight_time = (2 * v) / G;
    double pi = 3.141592653589793;
    double step = 0.01;
    double t = t_start;

    double a_radians = a * pi / 180;

    while (t <= t_end && t <= (int)total_flight_time)
    {
        double h = v * t * sin(a_radians) - 0.5 * G * t * t;
        double ratio = (double)t / total_flight_time;
        double R = 1.0 - ratio;

        double G = 0.0;
        double B = ratio;

        double d = v * t * cos(a_radians);

        double x = 50 + d;
        double y = 400 - h;

        set_pen_width(5);
        set_pen_color(R, G, B);
        draw_point((int)x, (int)y);

        t = t + step;
    }
}

```

```
}
```

```
void enemy_HQ(const int x, const int y)
```

```
{
```

```
    move_to(x,y);
```

```
    set_pen_width(1);
```

```
    draw_distance(80);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(80);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(80);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(80);
```

```
    turn_right_by_degrees(90);
```

```
    move_to(x + 10,y - 50);
```

```
    draw_distance(10);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(20);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(10);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(20);
```

```
    turn_right_by_degrees(90);
```

```
    move_to(x + 50,y - 50);
```

```
    draw_distance(10);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(20);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(10);
```

```
    turn_right_by_degrees(90);
```

```
    draw_distance(20);
```

```
    turn_right_by_degrees(90);
```

```
}
```

```
void enemy_defences(const int x, const int y)
```

```
{
```

```
    set_pen_width(5);
```

```
    move_to(x,y);
```

```
    draw_distance(160);
```

```
}
```

```
void main()
{
    interaction();
    double v = read_double();
    double a = read_double();
    double pi = 3.141592653589793;
    double a_radians = a * pi / 180;
    double total_flight_time = (2 * v) / G;

    //new game
    make_window(1000,400);
    set_pen_width(5);
    set_pen_color(color::red);
    draw_cannon(50,400,a_radians);
    enemy_defences(560,400);
    game(v+35, a+14.75, 0, 15);

    const int x = 700 + rand() % 250;
    const int y = 400;
    enemy_HQ(x,y);
}
```

